

Premiers pas avec Qt Quick-PyQt

Deuxième partie : création d'interfaces graphiques avec Qt Quick et interaction avec Python

Par [Charlie Gentil](#)  

Date de publication : 4 février 2015

TOUT PUBLIC

Durée : 40 min

La suite d'articles proposés a pour but d'initier le lecteur à la création d'une application écrite avec PyQt et Qt Quick.

Afin d'appréhender au mieux ce qui suit, des notions de Python et Qt Quick sont nécessaires. Vous trouverez tout ce qui peut vous être utile dans la page [cours Python](#) de Developpez.com et dans mon [précédent article sur Qt Quick](#).

De plus, une installation de PyQt 5.1 minimum est indispensable.

Commentez

En complément sur Developpez.com

- [Première utilisation de Qt Quick avec PyQt](#)
- [Configurer Qt Creator pour Python](#)

I - Introduction.....	3
II - Présentation de nouveaux modules Qt Quick.....	3
II-A - Création d'une fenêtre et de ses composants.....	3
II-B - Premiers pas avec les dispositions.....	4
III - Interaction avec un code Python.....	6
III-A - Lancement de la fenêtre depuis Python.....	6
III-B - Modification du code Python.....	6
III-C - Modification du code QML.....	7
IV - Conclusion.....	8
V - Remerciements.....	9

I - Introduction

L'**article précédent** expliquait la manière de générer une fenêtre avec des composants simples en QML dans le but de créer une première application graphique. Il est temps maintenant d'approfondir l'étude du langage QML et de découvrir la manière d'interagir entre la fenêtre et le code Python.

Les lignes de code qui suivent, dans un premier temps, auront pour objectif de créer une application graphique plus développée que celle vue dans le précédent article puis de la lancer depuis un script Python.

Dans un deuxième temps, elles montreront la manière de récupérer des valeurs de champs et aussi comment en envoyer.

L'exemple sera la création d'un simple programme d'identification d'utilisateur. Les fonctionnalités de base attendues seront donc :

- capture d'un identifiant ;
- capture d'un mot de passe ;
- vérification des informations saisies ;
- refus ou acceptation de la connexion.

Même si son emploi n'est pas indispensable, je vous conseille fortement l'utilisation de Qt Creator. En effet, cet EDI apporte deux avantages :



- depuis sa version 2.8, **il offre quelques fonctionnalités pour les développeurs Python** ;
- il est prévu pour fonctionner avec le langage QML, base de Qt Quick (coloration syntaxique, aide à la saisie et même un **Designer**).

II - Présentation de nouveaux modules Qt Quick

Le premier article montrait la création d'une interface graphique en utilisant QML et en créant chaque composant à partir de zéro : ainsi, créer un simple bouton nécessitait la création d'un rectangle, puis l'ajout de diverses propriétés en exploitant les composants de base de Qt Quick 2.

Heureusement, Qt Quick met à disposition beaucoup d'autres modules, dont certains fournissent des composants plus complets qu'un simple rectangle, par exemple.

Ces modules comptent notamment **Qt Quick Controls**, qui sera entre autres l'objet du chapitre suivant.

II-A - Création d'une fenêtre et de ses composants

Qt Quick Controls contient un certain nombre de composants courants comme la fenêtre principale de notre application, des boutons, des zones de texte, tout en restant dans la logique de Qt Quick, à savoir très personnalisables.

Dans le cadre du projet de cet article, une fenêtre de connexion, outre la fenêtre principale, il faudra une zone de saisie de l'identifiant, une zone de saisie du mot de passe, un bouton de validation et pour finir une zone de texte indiquant si la connexion est autorisée ou non.

Sans plus attendre, voici une manière d'écrire le code QML générant cette fenêtre :

Fenêtre de connexion

```
1. import QtQuick 2.4
2. import QtQuick.Controls 1.3
3. ApplicationWindow {
```

Fenêtre de connexion

```

4.
// Création de la fenêtre principale avec quelques attributs pratiques comme menuBar et statusBar
5.   title: qsTr("Fenêtre de connexion")
6.   width: 400
7.   height: 200
8.   menuBar: MenuBar {
9.       Menu {
10.          title: qsTr("&File")
11.          MenuItem {
12.              text: "Une action"
13.              onTriggered:
14.                  console.log("Magique cette barre de menu") // console.log(<msg>) affiche dans la console de sortie le message <m
15.              }
16.          }
17.      statusBar: StatusBar {
18.          Label { text: "QML est vraiment merveilleux" }
19.      }
20.      // Puis on crée et on place les composants souhaités.
21.      Label {
22.          text: "Login"
23.          x: 10
24.          y: 25
25.      }
26.      TextField {
27.          id: login
28.          x: 100
29.          y: 20
30.          placeholderText: qsTr("Enter your login")
31.      }
32.      Label {
33.          text: "Mot de passe"
34.          x: 10
35.          y: 55
36.      }
37.      TextField {
38.          id: password
39.          x: 100
40.          y: 50
41.          placeholderText: qsTr("Enter your password")
42.          echoMode: TextInput.Password
43.      }
44.      Button {
45.          text: "Se connecter"
46.          x: 10
47.          y: 100
48.      }
49.      Label {
50.          id: result
51.          x: 10
52.          y: 135
53.      }
54.  }
  
```

Le code proposé ne doit pas poser de problème de compréhension après le premier article. La **documentation officielle des composants QML** fournit plus de détails au besoin.

C'est bien joli tout ça, mais le placement des composants est loin d'être simple (positions absolues) et l'application est loin d'être très dynamique. À titre d'exemple, on ne redimensionne pas les widgets si la taille de la fenêtre change.

II-B - Premiers pas avec les dispositions

Heureusement pour nous, Qt Quick réserve encore de belles surprises comme les dispositions. Il en existe trois :

- GridLayout place ses composants dans une grille ;
- ColumnLayout place ses composants en colonne ;

- RowLayout place ses composants en ligne.

Ces dispositions sont fournies par le module Qt Quick Layouts. GridLayout est certes la plus complexe à utiliser, mais également la plus adaptée à la situation.

Voici le code ci-dessus retravaillé avec GridLayout :

Utilisation des dispositions

```

1. import QtQuick 2.4
2. import QtQuick.Controls 1.3
3. import QtQuick.Layouts 1.1
4. ApplicationWindow {
5.     title: qsTr("Fenêtre de connexion")
6.     width: 400
7.     height: 200
8.     menuBar: MenuBar {
9.         Menu {
10.            title: qsTr("&File")
11.            MenuItem {
12.                text : "Une action"
13.                onTriggered: console.log("Magique cette barre de menu")
14.            }
15.        }
16.    }
17.    statusBar: StatusBar {
18.        Label { text: "QML est vraiment merveilleux" }
19.    }
20.    GridLayout {
21.        id: grid
22.        anchors.fill: parent // Ancrer la disposition au composant souhaité
23.        columns: 2           // Nombre de colonnes
24.        anchors.margins: 5   // Marge entre la disposition et les bords de son conteneur
25.        columnSpacing: 10    // Espace entre chaque colonne
26.        Label {
27.            text: "Identifiant"
28.        }
29.        TextField {
30.            id: login
31.            Layout.fillWidth: true
32.            placeholderText: qsTr("Enter your login")
33.        }
34.        Label {
35.            text: "Mot de passe"
36.        }
37.        TextField {
38.            id: password
39.            Layout.fillWidth: true
40.            placeholderText: qsTr("Enter your password")
41.            echoMode: TextInput.Password
42.        }
43.        Button {
44.            text: "Se connecter"
45.            // Ces deux lignes autorisent le redimensionnement automatique du composant
46.            Layout.fillWidth: true
47.            Layout.fillHeight: true
48.            Layout.columnSpan: 2 // Étale sur deux colonnes
49.        }
50.        Label {
51.            id: result
52.            text : "Non connecté"
53.            Layout.fillWidth: true
54.            Layout.columnSpan: grid.columns // Étale sur toutes les colonnes
55.        }
56.    }
57. }
```

Ah... enfin un résultat simple, dynamique et très pratique. Il ne reste plus qu'à rendre le code fonctionnel en interagissant avec un code Python.



Une autre solution aurait été d'affecter un pourcentage de la hauteur et/ou de la largeur de la fenêtre à chaque composant en utilisant leurs propriétés **height** ou **width**. N'hésitez pas à faire des essais si cette solution vous intéresse.

III - Interaction avec un code Python

Maintenant que l'application est prête d'un point de vue graphique, il est temps de la rendre fonctionnelle par Python.

III-A - Lancement de la fenêtre depuis Python

Heureusement, cette intégration se fait très simplement par les lignes de code suivantes :

Exécution depuis Python

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3. import sys
4. from PyQt5.QtWidgets import QApplication
5. from PyQt5.QtQml import QQmlApplicationEngine
6.
7. if __name__ == "__main__":
8.     app = QApplication(sys.argv)
9.     engine = QQmlApplicationEngine()
10.    engine.load('test.qml')
11.    win = engine.rootObjects()[0]
12.    win.show()
13.    sys.exit(app.exec_())
```

Ce script est assez simple : il crée un objet QQmlApplicationEngine auquel un document QML est associé et affiché. En lançant ce code, la fenêtre attendue doit apparaître.

Très bien ! Après l'affichage d'une fenêtre créée en QML via Python, il reste maintenant à interagir pleinement avec elle depuis Python. Interaction avec le code.

Ce chapitre présente trois manières de créer une interaction entre du code QML et du code Python. Ces trois méthodes sont équivalentes : le choix de l'une ou l'autre se porte sur la situation ou les habitudes.



Il y aura des modifications à apporter aussi bien dans la partie Python que QML. Les éléments nouveaux seront expliqués au fur et à mesure.

III-B - Modification du code Python

Dans un premier temps, le code Python de lancement de l'application est modifié pour passer au contexte Qt Quick d'une instance de la classe MainApp (à créer) qui implémente la partie Python de la communication.

Modification du code Python

```
1. if __name__ == "__main__":
2.     app = QApplication(sys.argv)
3.     engine = QQmlApplicationEngine()
4.     # Création d'un objet QQmlContext pour communiquer avec le code QML
5.     ctx = engine.rootContext()
6.     engine.load('test.qml')
7.     win = engine.rootObjects()[0]
8.     py_mainapp = MainApp(ctx, win)
9.     ctx.setContextProperty("py_MainApp", py_mainapp)
10.    win.show()
11.    sys.exit(app.exec_())
```

Une fois ceci fait, il reste à créer la classe MainApp et à y mettre les fonctions nécessaires pour la communication. La première méthode consiste à renvoyer la réponse par une propriété du contexte global : pour y accéder en QML, il suffira de taper le nom de la propriété.

Classe MainApp

```

1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3. import sys
4. from PyQt5.QtCore import QObject, pyqtSlot, QVariant
5.
6. # Classe servant dans l'interaction.
7. class MainApp(QObject):
8.     def __init__(self, context, parent=None):
9.         super(MainApp, self).__init__(parent)
10.         # Recherche d'un enfant appelé myButton dont le signal clicked sera connecté à la fonction test3
11.         self.win = parent
12.         self.win.findChild(QObject, "myButton").clicked.connect(self.test3)
13.         self.ctx = context
14.
15.         # Création de la fonction d'identification
16.         def verifConnection(self, login, password):#voir PEP8
17.             if login == "login" and password == "mdp":
18.                 return "Bonjour %s. Vous êtes maintenant connecté" % login
19.             else:
20.                 return "Désolé, authentification impossible"
21.
22.         # Premier test de communication : propriétés contextuelles.
23.         @pyqtSlot(QVariant, QVariant)
24.         def test1(self, login, password):
25.             txt = self.verifConnection(login, password)
26.             # Transmission du résultat comme une propriété nommée retour
27.             self.ctx.setProperty("retour", txt)
28.             return 0
29.
30.         # Deuxième test de communication : création d'une fonction ayant pour type de retour un QVariant (obligat
31.         @pyqtSlot(QVariant, QVariant, result=QVariant)
32.         def test2(self, login, password):
33.             return self.verifConnection(login, password)
34.
35.         # Troisième test de communication : modification directe d'un composant QML.
36.         def test3(self):
37.             # Recherche des enfants par leur attribut objectName, récupération de la valeur de leur propriété tex
38.             login = self.win.findChild(QObject, "myLogin").property("text")
39.             password = self.win.findChild(QObject, "myPassword").property("text")
40.             txt = self.verifConnection(login, password)
41.             self.win.findChild(QObject, "labelCo").setProperty("text", txt)
42.             return 0

```

III-C - Modification du code QML

Maintenant que notre code Python est prêt, il ne reste plus qu'à modifier légèrement le code QML pour exploiter ces moyens de communication.

Modification code QML

```

1. import QtQuick 2.4
2. import QtQuick.Controls 1.3
3. import QtQuick.Layouts 1.1
4. ApplicationWindow {
5.     title: qsTr("Fenêtre de connexion")
6.     width: 400
7.     height: 200
8.     menuBar: MenuBar {
9.         Menu {
10.            title: qsTr("&File")
11.            MenuItem {
12.                text : "Une action"
13.                onTriggered: console.log("Magique cette barre de menu")
14.            }

```

Modification code QML

```

15.     }
16. }
17. statusBar: StatusBar {
18.     Label { text: "QML est vraiment merveilleux" }
19. }
20. GridLayout {
21.     id: grid
22.     anchors.fill: parent
23.     columns: 2
24.     anchors.margins: 5
25.     columnSpacing: 10
26.     Label {
27.         text: "Identifiant"
28.     }
29.     TextField {
30.         id: login
31.         objectName: "myLogin" // test3() cherche un enfant nommé myLogin
32.         Layout.fillWidth: true
33.         placeholderText: qsTr("Enter your login")
34.     }
35.     Label {
36.         text: "Mot de passe"
37.     }
38.     TextField {
39.         id: password
40.         objectName: "myPassword"
41.         Layout.fillWidth: true
42.         placeholderText: qsTr("Enter your password")
43.         echoMode: TextInput.Password
44.     }
45.     Button {
46.         text: "Se connecter par test1()"
47.         Layout.fillWidth: true
48.         Layout.fillHeight: true
49.         Layout.columnSpan: 2
50.         onClicked: {
51.             py_MainApp.test1(login.text, password.text)
52.             result.text = retour
53.         }
54.     }
55.     Button {
56.         text: "Se connecter par test2()"
57.         Layout.fillWidth: true
58.         Layout.fillHeight: true
59.         Layout.columnSpan: 2
60.         onClicked: result.text = py_MainApp.test2(login.text, password.text)
61.     }
62.     Button {
63.         text: "Se connecter par test3()"
64.         objectName : "myButton"
65.         Layout.fillWidth: true
66.         Layout.fillHeight: true
67.         Layout.columnSpan: 2
68.     }
69.     Label {
70.         id: result
71.         objectName: "labelCo"
72.         text : "Non connecté"
73.         Layout.fillWidth: true
74.         Layout.columnSpan: grid.columns
75.     }
76. }
77. }

```

IV - Conclusion

Après une première partie destinée à découvrir Qt Quick, cette deuxième montre comment créer une interface graphique très fonctionnelle avec un minimum de code ainsi que l'interaction entre du code Python et du code QML.

La lecture de ces deux premières parties doit vous permettre d'imaginer sereinement vos prochaines lignes de code. Cependant, Qt Quick possède encore beaucoup d'autres fonctionnalités qui seront abordées dans la troisième et dernière partie.

V - Remerciements

Je tiens tout particulièrement à remercier **Thibaut Cuvelier** et **Claude LELOUP** pour leur relecture attentive.